

CHƯƠNG 8

QUẢN LÝ LOG & CẢNH BÁO SỰ CỐ

Mục tiêu chương

Sau khi học xong chương này, người học có thể:

- Giải thích tầm quan trọng của quản lý log tập trung.
- Trình bày ý nghĩa của log và các hệ thống thu thập, xử lý log.
- Triển khai hệ thống để thu thập và quản lý log tập trung.
- Thiết kế Alerting Rules và cấu hình Alertmanager.
- Kết hợp Monitoring + Logging để vận hành hệ thống hiệu quả.

8.1 Vai trò của log trên hệ thống

Trong môi trường CNTT hiện đại, hệ thống server, ứng dụng và network liên tục tạo ra lượng dữ liệu log khổng lồ mỗi ngày. Nếu không có cơ chế quản lý phù hợp, doanh nghiệp sẽ rất khó theo dõi hoạt động hệ thống, phát hiện lỗi hoặc xử lý sự cố kịp thời. Đây cũng là lý do Log Server trở thành thành phần cực kỳ quan trọng trong vận hành hạ tầng công nghệ hiện nay.

Log (nhật ký) hệ thống đóng vai trò như "hộp đen" ghi lại toàn bộ hoạt động, sự kiện và lỗi xảy ra trên ứng dụng hoặc hạ tầng. Log Server là hệ thống dùng để thu thập, lưu trữ, quản lý và phân tích log từ nhiều nguồn khác nhau như server, application, database, network device hoặc cloud service. Thay vì lưu log phân tán trên từng máy chủ riêng lẻ, Log Server giúp tập trung toàn bộ log về một nơi nhằm hỗ trợ monitoring, troubleshooting và security analysis hiệu quả hơn.

Trong môi trường doanh nghiệp hiện đại, số lượng hệ thống và ứng dụng có thể lên đến hàng trăm hoặc hàng nghìn service hoạt động cùng lúc. Nếu quản trị viên phải truy cập từng server để kiểm tra log thủ công thì việc phân tích lỗi hoặc điều tra sự cố sẽ cực kỳ mất thời gian. Log Server giải quyết vấn đề này bằng cách xây dựng cơ chế centralized logging tập trung cho toàn bộ hệ thống.

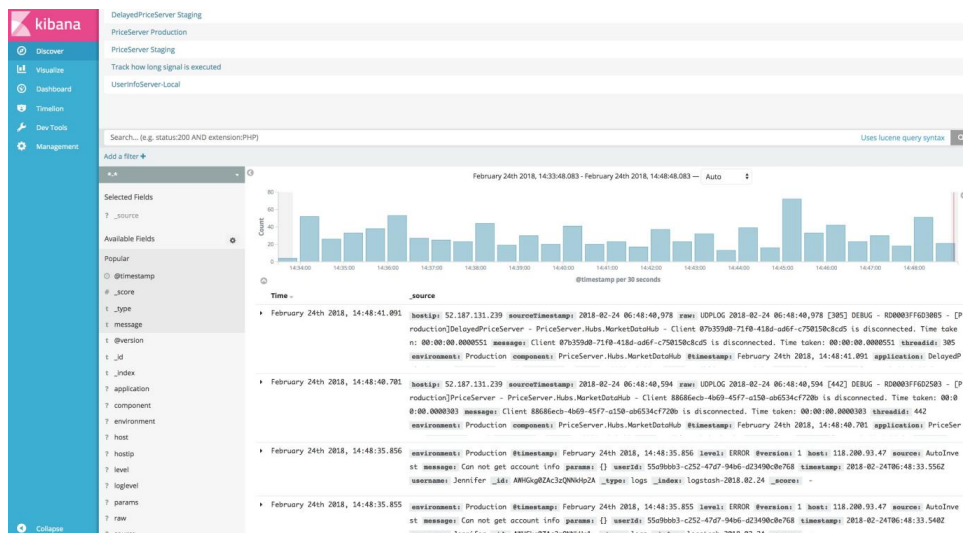
Log Server không chỉ đơn thuần lưu trữ dữ liệu log mà còn hỗ trợ search, filtering, alerting và visualization theo thời gian thực. Điều này giúp DevOps Engineer, System Administrator hoặc Security Team nhanh chóng phát hiện bất thường và xử lý sự cố trước khi ảnh hưởng lớn đến hệ thống.

Ngày nay, cùng với sự phát triển của microservices, **Kubernetes** và Cloud Computing, vai trò của Log Server ngày càng trở nên quan trọng hơn bao giờ hết. Đây gần như là thành phần bắt buộc trong các hệ thống cloud-native hoặc **DevOps** hiện đại.

Đây là công cụ thiết yếu giúp lập trình viên và quản trị viên truy vết sự cố, đảm bảo bảo mật, và tối ưu hóa hiệu suất.

8.1.1 Tại sao cần thu thập log

Vai trò lớn nhất của Log Server là hỗ trợ giám sát hoạt động hệ thống theo thời gian thực. Thông qua log được thu thập từ nhiều nguồn khác nhau, doanh nghiệp có thể theo dõi trạng thái server, application và network để phát hiện sớm các dấu hiệu bất thường.



Hình 8.1: Quản lý log trong Kibana

Trong quá trình vận hành hệ thống, lỗi phần mềm hoặc sự cố hạ tầng là điều gần như không thể tránh khỏi. Khi xảy ra vấn đề, Log Server giúp kỹ sư nhanh chóng truy vết nguyên nhân bằng cách tìm kiếm log liên quan thay vì kiểm tra thủ công trên từng máy chủ riêng lẻ. Điều này giúp rút ngắn đáng kể thời gian troubleshooting và giảm downtime cho hệ thống.

Ngoài monitoring và troubleshooting, Log Server còn đóng vai trò rất quan trọng trong bảo mật hệ thống. Security Team có thể sử dụng log để phát hiện hành vi đăng nhập bất thường, truy cập trái phép hoặc dấu hiệu tấn công mạng nhằm phản ứng kịp thời trước các nguy cơ bảo mật.

Trong nhiều doanh nghiệp lớn, Log Server còn hỗ trợ compliance và audit. Các tiêu chuẩn như PCI DSS, ISO 27001 hoặc GDPR thường yêu cầu tổ chức phải lưu trữ và theo dõi log hệ thống nhằm phục vụ điều tra sự cố và kiểm tra bảo mật.

Việc thu thập log không chỉ là một thói quen kỹ thuật tốt mà còn là yêu cầu bắt buộc về mặt pháp lý. Tại Việt Nam, Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân đặt ra những yêu cầu khắt khe mà doanh nghiệp phải tuân thủ trong việc lưu trữ và bảo vệ dữ liệu, bao gồm cả các vết log truy cập. Thiếu khả năng giám sát log đồng nghĩa với việc doanh nghiệp đang "mù" trước các dấu hiệu tấn công sớm, mã độc hoặc hành vi xâm nhập trái phép.

Về mặt vận hành, thu thập log giúp tối ưu hóa thời gian khắc phục sự cố (giảm thiểu chỉ số MTTR - Mean Time to Repair). Thay vì phải đăng nhập vào từng máy chủ để tìm lỗi một cách thủ công, người quản trị có thể truy vết nguyên nhân gốc rễ (root cause) thông qua hệ thống log tập trung, từ đó giảm thiểu đáng kể thời gian gián đoạn dịch vụ (downtime). Ngoài ra, việc phân tích log giúp phát hiện các "điểm nghẽn" hiệu suất, các ứng dụng hoạt động kém hiệu quả hoặc tài nguyên đang bị lãng phí.

Trong lĩnh vực bảo mật, log monitoring là "lá chắn" giúp phát hiện sớm các cuộc

tấn công Brute Force, rò rỉ dữ liệu hoặc các hành vi bất thường của người dùng nội bộ. Một hệ thống thu thập log tốt sẽ cung cấp dữ liệu cho các trung tâm điều hành an ninh (SOC) để phân tích và phản ứng kịp thời trước các nguy cơ tấn công mạng ngày càng tinh vi. Cuối cùng, dữ liệu log còn là cơ sở quan trọng để các nhà quản lý đưa ra quyết định cải tiến hệ thống dựa trên thông tin thực tế thay vì cảm tính.

8.1.2 Các loại log phổ biến

Hệ thống CNTT hiện đại tạo ra rất nhiều loại log khác nhau, mỗi loại phục vụ một mục đích cụ thể trong việc giám sát và quản trị.

- **System Log (nhật ký hệ thống):** ghi lại hoạt động của hệ điều hành và các thành phần hạ tầng như quá trình khởi động (boot), lỗi kernel, hoạt động của CPU/RAM hoặc lỗi phần cứng. Trên Linux, các log này thường được quản lý bởi `rsyslog` hoặc `systemd-journald` và lưu tại thư mục `/var/log`.
- **Application Log (nhật ký ứng dụng):** chứa thông tin về các hoạt động của phần mềm, lỗi API, giao dịch cơ sở dữ liệu hoặc các ngoại lệ (exceptions) trong mã nguồn. Đây là loại log cực kỳ quan trọng đối với các lập trình viên và DevOps Engineer để tối ưu hóa ứng dụng.
- **Security Log (nhật ký bảo mật):** tập trung vào các sự kiện xác thực như đăng nhập thành công/thất bại (Event ID 4624/4625 trên Windows), thay đổi quyền truy cập, hoặc thay đổi chính sách bảo mật. Đây là nguồn dữ liệu sống còn cho việc điều tra sự cố an ninh.
- **Access Log (nhật ký truy cập):** ghi nhận thông tin người dùng tương tác với hệ thống, ví dụ log của Web Server (Nginx, Apache) lưu lại địa chỉ IP, URL được yêu cầu, và mã trạng thái HTTP (200, 404, 500).
- **Network Log (nhật ký mạng):** lưu lại lưu lượng qua tường lửa (firewall), hoạt động của router, switch giúp phát hiện các hành vi quét cổng (port scanning) hoặc tấn công từ chối dịch vụ (DDoS).

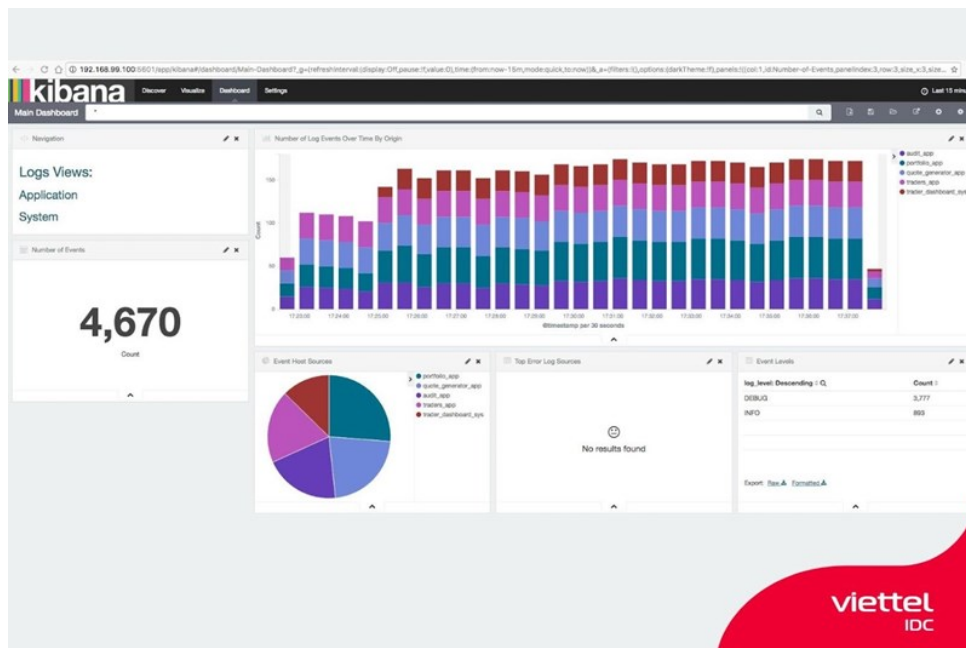
Hiểu rõ cấu trúc log cũng rất quan trọng. Ví dụ, tiêu chuẩn Syslog sử dụng mã cơ sở (Facility Codes) để phân loại nguồn phát (như `kern`, `user`, `mail`, `auth`) và mức độ nghiêm trọng (Severity Levels) từ 0 (Emergency – khẩn cấp nhất) đến 7 (Debug – gỡ lỗi).

Log Server hoạt động như thế nào? Log Server hoạt động dựa trên mô hình centralized logging, nghĩa là toàn bộ log từ nhiều nguồn khác nhau sẽ được gửi về một hệ thống tập trung để quản lý và phân tích. Quá trình đầu tiên là log collection. Các agent hoặc service như Fluentd, Filebeat hoặc Logstash sẽ thu thập log từ server, application hoặc network device rồi chuyển tiếp đến Log Server.

Sau khi nhận log, hệ thống sẽ thực hiện parsing nhằm chuẩn hóa dữ liệu và tách các field quan trọng như timestamp, severity level hoặc source IP. Điều này giúp việc search và filtering hiệu quả hơn rất nhiều.

Tiếp theo, log sẽ được indexing và lưu trữ trong storage backend như Elasticsearch hoặc database chuyên dụng. Việc indexing giúp người dùng tìm kiếm log cực nhanh dù dữ liệu có thể lên đến hàng terabyte hoặc petabyte.

Ngoài lưu trữ, Log Server còn hỗ trợ dashboard visualization nhằm hiển thị dữ liệu trực quan dưới dạng graph hoặc chart. Điều này giúp quản trị viên theo dõi trạng thái hệ thống realtime dễ dàng hơn.



Hình 8.2: Bảng dashboard hiển thị log trên Kibana

Ưu điểm lớn nhất của Log Server là khả năng quản lý log tập trung. Điều này giúp giảm đáng kể thời gian kiểm tra và troubleshooting hệ thống. Ngoài ra, Log Server giúp tăng khả năng giám sát realtime cho toàn bộ infrastructure. DevOps Team có thể nhanh chóng phát hiện lỗi hoặc sự cố mà không cần truy cập từng server riêng lẻ.

Trong lĩnh vực bảo mật, Log Server hỗ trợ phân tích security event hiệu quả hơn rất nhiều so với log phân tán truyền thống. Security Team có thể correlation nhiều nguồn log nhằm phát hiện tấn công mạng nhanh hơn.

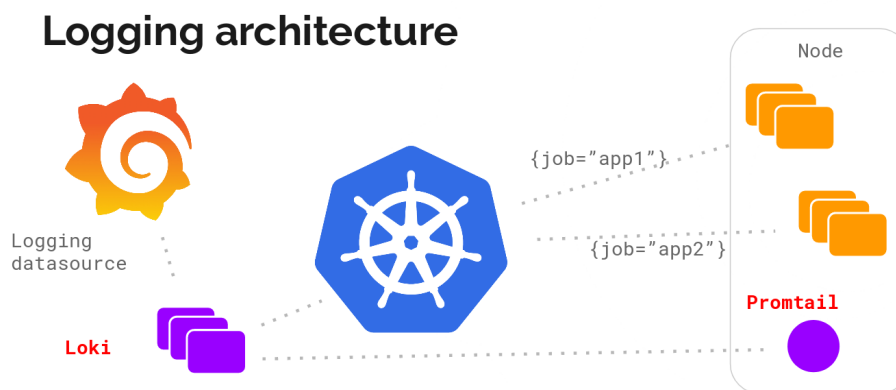
Ngoài ra, Log Server còn hỗ trợ compliance và audit hiệu quả hơn nhờ khả năng lưu trữ log tập trung lâu dài. Khả năng search và filtering mạnh mẽ cũng giúp tiết kiệm rất nhiều thời gian điều tra sự cố. Thay vì đọc hàng triệu dòng log thủ công, kỹ sư có thể tìm kiếm theo keyword hoặc timestamp chỉ trong vài giây.

8.1.3 Giải pháp quản lý log tập trung

Quản lý log tập trung là quá trình thu thập toàn bộ dữ liệu log từ nhiều nguồn phân tán (máy chủ, ứng dụng, thiết bị cloud) về một nền tảng duy nhất để quản lý và phân tích. Trong các hệ thống lớn với hàng trăm microservices, việc truy cập từng server để kiểm tra log thủ công là bất khả thi; CLM chính là lời giải cho thách thức này.

Trên thị trường hiện có nhiều giải pháp nổi bật, chia thành hai nhóm chính: mã nguồn mở và thương mại.

- **Elastic Stack (ELK):** gồm Elasticsearch, Logstash, Kibana. Đây là giải pháp rất mạnh mẽ, linh hoạt, có cộng đồng hỗ trợ lớn nhưng yêu cầu tài nguyên phần cứng cao (ngốn RAM) và kỹ năng vận hành phức tạp.
- **Splunk:** ”gã khổng lồ” trong lĩnh vực phân tích log với khả năng trực quan hóa và bảo mật tuyệt vời, nhưng chi phí bản quyền rất cao, thường chỉ phù hợp cho các tập đoàn lớn.
- **Grafana Loki:** được mệnh danh là ”Prometheus cho logs”. Điểm khác biệt lớn



Hình 8.3: Kiến trúc log

nhất là Loki không lập chỉ mục nội dung log mà chỉ lập chỉ mục các nhãn (labels), giúp tiết kiệm tài nguyên lưu trữ và CPU một cách kinh ngạc (có thể giảm 10–50 lần chi phí so với ELK).

- **digiLogs:** một giải pháp chuyên sâu cho doanh nghiệp Việt từ TPISoftware, tập trung vào hiệu suất cao (hơn 10.000 sự kiện/giây) và khả năng tuân thủ các quy định bảo mật tại địa phương.

Lợi ích của CLM không chỉ dừng lại ở kỹ thuật mà còn mang lại giá trị kinh tế. Theo báo cáo từ Gartner, 80% doanh nghiệp sử dụng CLM giảm được 30% thời gian xử lý sự cố.

8.1.4 Xử lý và phân tích log

Quá trình xử lý log trong một hệ thống tập trung thường trải qua các giai đoạn nghiêm ngặt để biến dữ liệu thô thành thông tin có giá trị:

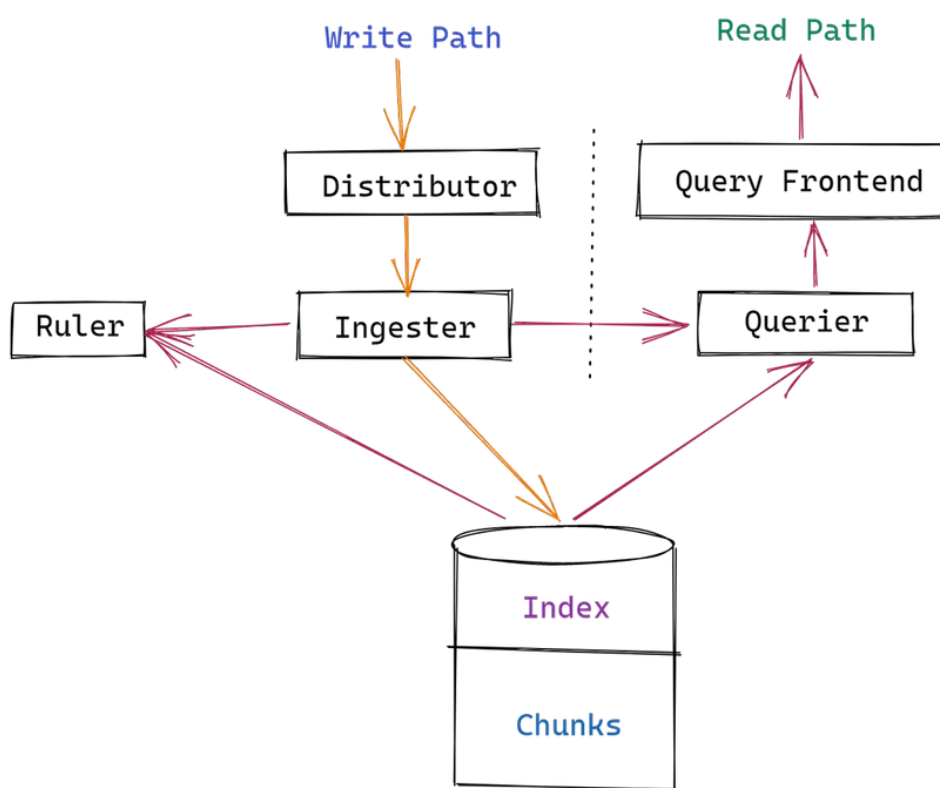
- **Thu thập (Collection):** sử dụng các agent nhẹ như Fluentd, Filebeat hoặc Promtail để "nhặt" log từ file local hoặc stream và đẩy về server trung tâm.
- **Chuẩn hóa và parsing (Normalization):** dữ liệu log từ các nguồn khác nhau thường không đồng nhất về định dạng. Hệ thống cần thực hiện phân tách các trường (fields) như timestamp, mức độ lỗi, IP nguồn để dễ dàng tìm kiếm và lọc dữ liệu sau này.
- **Lưu trữ và indexing:** log được nén để tiết kiệm không gian và đánh chỉ mục. Với các hệ thống hiện đại như Loki, việc lưu trữ thường được đẩy ra các Object Storage giá rẻ như AWS S3 hoặc MinIO.
- **Phân tích và trực quan hóa:** sử dụng các công cụ như Grafana để tạo dashboard, vẽ biểu đồ xu hướng lỗi theo thời gian thực giúp người quản trị có cái nhìn toàn cảnh về hệ thống.
- **Truy vấn (Querying):** khả năng tìm kiếm nhanh chóng trên khối lượng dữ liệu khổng lồ (hàng terabyte) là yếu tố quyết định. Ví dụ, ngôn ngữ LogQL của Loki cho phép chúng ta lọc lỗi theo từng pod hoặc namespace trong môi trường Kubernetes một cách dễ dàng.

8.2 Triển khai hệ thống thu thập log tập trung

8.2.1 Cấu hình và triển khai Loki

Grafana Loki được thiết kế theo kiến trúc microservices giúp nó có khả năng mở rộng cực tốt. Để triển khai Loki hiệu quả, cần hiểu rõ 4 thành phần cốt lõi của nó:

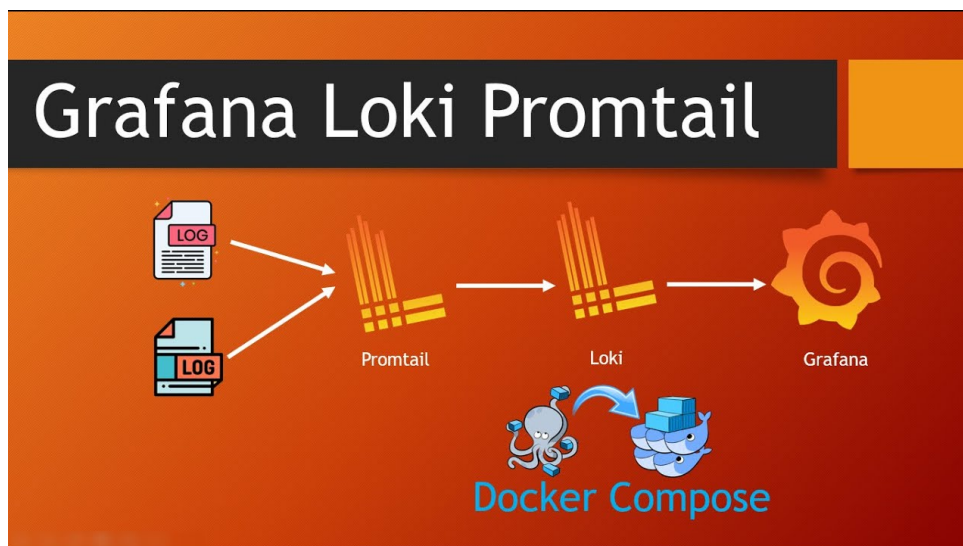
- **Distributor:** tiếp nhận log từ các agent (như Promtail), xác thực và chia nhỏ dữ liệu để gửi đến các Ingester.
- **Ingester:** chịu trách nhiệm ghi dữ liệu vào các storage dài hạn và giữ log trong bộ nhớ đệm để phục vụ các truy vấn tức thời.
- **Querier:** xử lý các câu lệnh truy vấn từ người dùng, tìm kiếm dữ liệu cả trong bộ nhớ đệm của Ingester và trong storage dài hạn.
- **Query Frontend:** một thành phần tùy chọn giúp tăng tốc truy vấn bằng cách chia nhỏ các yêu cầu lớn và thực hiện song song.



Hình 8.4: Các thành phần của Grafana Loki

Quy trình triển khai cơ bản (sử dụng Docker):

- Chuẩn bị môi trường: cài đặt Docker và Docker Compose trên server (Ubuntu/Debian).
- Cấu hình Loki (`loki-config.yaml`): thiết lập các thông số về port (mặc định 3100), nơi lưu trữ dữ liệu (local file hoặc S3), và chính sách lưu trữ (retention).
- Triển khai Promtail: đây là agent được cài đặt trên từng máy chủ cần lấy log. Trong file `promtail-config.yaml`, ta cần phải chỉ rõ địa chỉ URL của Loki server và các đường dẫn file log cần theo dõi (ví dụ: `/var/log/*.log`).
- Kết nối Grafana: sau khi Loki chạy, vào Grafana Dashboard → Configuration → Data Sources → Add "Loki" và nhập URL của Loki server (ví dụ: <http://loki:3100>).



Hình 8.5: Các thành phần trong quản lý log

Để cấu hình các dịch vụ mới vào docker compose từ bài lab số 7, ta cần chỉnh sửa file cấu hình docker như sau:

Tập tin docker-compose.yml được bổ sung:

```
services:
  # ... (giữ nguyên wordpress, mysql, prometheus, grafana, cadvisor, mysql-exporter...)

  loki:
    image: grafana/loki:latest
    container_name: loki
    restart: unless-stopped
    ports:
      - "3100:3100"
    volumes:
      - ./loki-config.yml:/etc/loki/loki-config.yml
      - loki_data:/loki
    command: -config.file=/etc/loki/loki-config.yml
    networks:
      - monitoring

  promtail:
    image: grafana/promtail:latest
    container_name: promtail
    restart: unless-stopped
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - ./promtail-config.yml:/etc/promtail/config.yml
      - /var/lib/docker/containers:/var/lib/docker/containers:ro
    command: -config.file=/etc/promtail/config.yml
    networks:
      - monitoring
    depends_on:
      - loki

volumes:
  loki_data:
```

```
# ... các volume cũ
```

Để cấu hình Loki, chúng ta cần tạo file `loki-config.yml` và đặt trong cùng thư mục với file `compose`. Tập tin cấu hình này chứa các thông tin thiết lập cho dịch vụ để thực hiện lưu log hệ thống. Nội dung file như dưới đây:

Tập tin `loki-config.yml`:

```
auth_enabled: false

server:
  http_listen_port: 3100

common:
  path_prefix: /loki
  storage:
    filesystem:
      chunks_directory: /loki/chunks
      rules_directory: /loki/rules
  replication_factor: 1
  ring:
    kvstore:
      store: inmemory

schema_config:
  configs:
    - from: "2024-01-01"
      store: boltdb-shipper
      object_store: filesystem
      schema: v12
      index:
        prefix: index_
        period: 24h
```

Tiếp theo, ta cần tạo file cấu hình cho Promtail như sau:

Tạo file `promtail-config.yml`:

```
server:
  http_listen_port: 9080
  grpc_listen_port: 0

positions:
  filename: /tmp/positions.yaml

clients:
  - url: http://loki:3100/loki/api/v1/push

scrape_configs:
  - job_name: docker
    docker_sd_configs:
      - host: unix:///var/run/docker.sock
        refresh_interval: 15s
    relabel_configs:
      - source_labels: ['__meta_docker_container_name']
        regex: '/(.*)'
        target_label: container
```

Chương 8. Quản lý Log & Cảnh báo sự cố

```
- source_labels: ['__meta_docker_container_label_com_docker_compose_service']  
  target_label: service
```

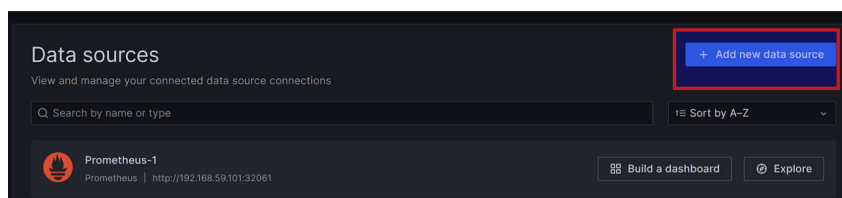
Sau khi hoàn thiện quá trình thiết lập và cấu hình, ta thực hiện khởi động các container bằng câu lệnh:

```
$docker compose up -d
```

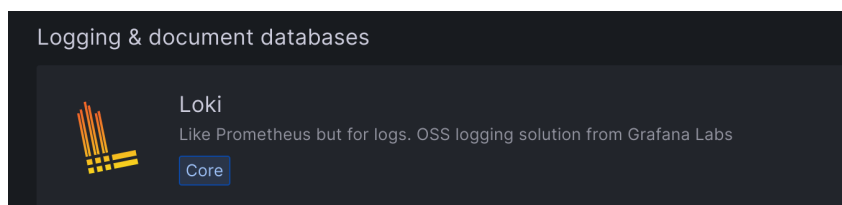
Kiểm tra và đảm bảo các container đã khởi động và expose ra đúng cổng dịch vụ như thiết lập trong file docker compose ở trên.

Tiếp theo, ta cần kết nối Loki vào Grafana. Truy cập dịch vụ Grafana thường chạy trên địa chỉ **http://localhost:3000**.

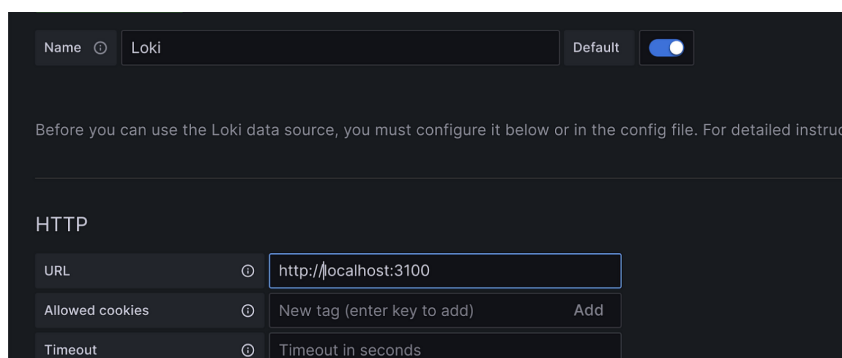
- Vào menu Configuration → Data Sources → Add data source → chọn Loki.
- Điền các thông số sau đây: Name: **Loki**; URL: **http://loki:3100**; Save & Test.



Hình 8.6: Thêm mới data source trong Grafana



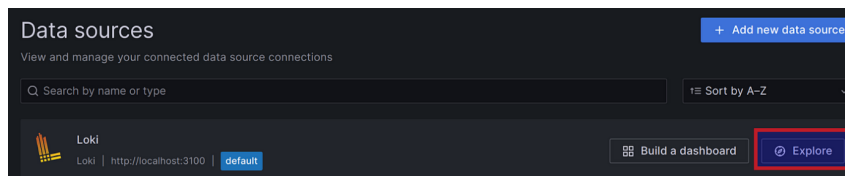
Hình 8.7: Lựa chọn Loki như nguồn cung cấp log



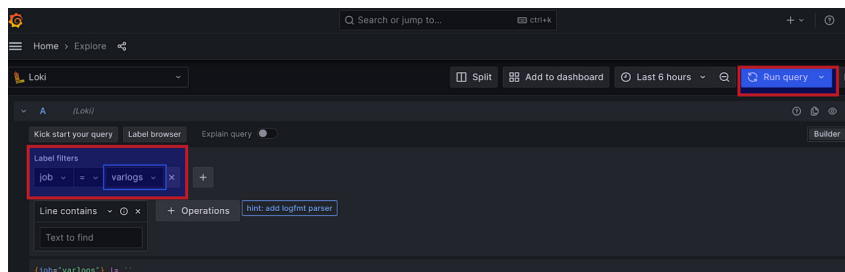
Hình 8.8: Thiết lập địa chỉ kết nối tới Loki

Sau khi thêm xong, ta có thể sử dụng Explore để query log theo `container="wordpress"` hoặc `service="mysql"`. Tạo dashboard kết hợp Log + Metrics.

Lưu ý quan trọng:



Hình 8.9: Thiết lập thành công



Hình 8.10: Thiết lập logfile và Run Query

- Dịch vụ Promtail đang dần EOL → khuyến nghị dùng Grafana Alloy thay thế cho production (một agent duy nhất cho cả metrics + logs).
- Mount Docker socket giúp Promtail tự động phát hiện tất cả container.
- Ta có thể lọc log theo label (container name, service name, environment...).
- Để lưu log lâu dài, sau này có thể chuyển sang S3/Object Storage thay vì filesystem.

8.2.2 Thực hiện truy vấn log trên Loki

Một số query Loki LogQL hay dùng cho WordPress & MySQL. LogQL là ngôn ngữ truy vấn của Loki, tương tự PromQL, dưới đây là một số câu truy vấn thường dùng: Query chung cho WordPress:

Bảng 8.1: Một số query LogQL cho WordPress (giữ nguyên nội dung gốc, một số ô bị cắt cụt trong tài liệu DOCX)

Mục đích	LogQL Query	Giải thích
Toàn bộ log của WordPress	<code>{container="wordpress"}</code>	Xem tất cả log
Lỗi và cảnh báo	<code>`{container="wordpress"}</code>	<code>= "error" or</code>
Access log (truy cập trang)	<code>`{container="wordpress"}</code>	<code>= "GET" or</code>
Request chậm (> 1 giây)	<code>`{container="wordpress"}</code>	<code>= "ms" regexp(\d+)ms` duration > 1s</code>
Theo IP truy cập	<code>{container="wordpress"}</code> <code> regexp</code> <code>(\d{1,3}.\d{1,3}.\d{1,3}.\d{1,3})</code>	Lọc theo IP
Login thành công / thất bại	<code>`{container="wordpress"}</code> <code> = "login" or</code>	<code>= "wp-login" `</code>

Query cho MySQL:

Query kết hợp (rất hay sử dụng):

- Log của cả 2 service trong 15 phút gần nhất:

Bảng 8.2: Một số query LogQL cho MySQL (giữ nguyên nội dung gốc, một số ô bị cắt cụt trong tài liệu DOCX)

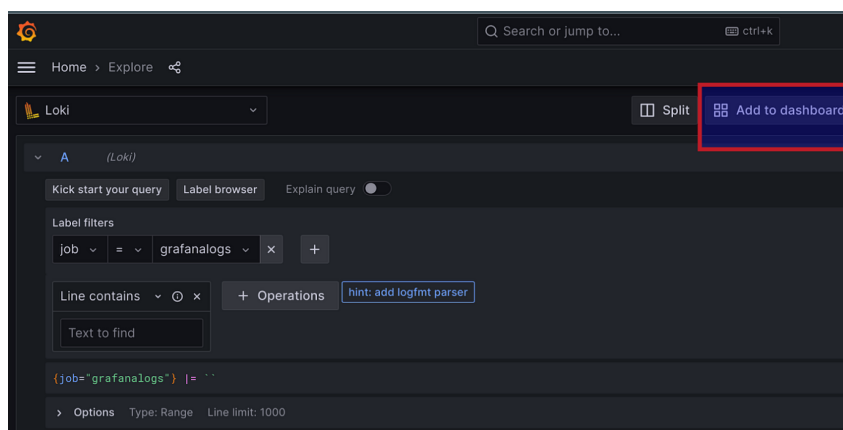
Mục đích	LogQL Query
Toàn bộ log MySQL	<code>{container="mysql"}</code>
Error & Warning	<code>`{container="mysql"}</code>
Query chậm	<code>{container="mysql"} = "slow" or = "long query"</code>
Kết nối bị từ chối	<code>{container="mysql"} = "Access denied"</code>
Theo thời gian cụ thể	<code>{container="mysql"} = "2025" (thay năm hiện tại)</code>

```
{container=~"wordpress|mysql"} |= "error"
```

- Xem log theo service name (từ label):

```
{service="wordpress"} or {service="mysql"}
```

Lưu ý: trong Grafana Explore, chọn Loki → viết query → chuyển sang Table hoặc Logs view.



Hình 8.11: Thêm Dashboard trong Loki để hiển thị log

Dashboard khuyến nghị nên tạo thủ công (kết hợp Metrics + Logs). Tạo 1 dashboard chính với các panel sau:

- Top 10 request chậm (Log panel).
- Số lỗi theo thời gian (Stat + Time series).
- Log WordPress realtime (Logs panel).
- CPU / Memory WordPress (từ Prometheus).
- MySQL Connections (từ Prometheus).

8.2.3 Tích hợp Alertmanager vào hệ thống để vận hành hiệu quả

Alertmanager là thành phần quan trọng của Prometheus stack, chịu trách nhiệm xử lý alert, gom nhóm (grouping), khử trùng lặp (deduplication), và gửi thông báo (email, Slack, Telegram, Discord...).

Để thêm Alertmanager vào hệ thống đang vận hành, ta cần chỉnh sửa file cấu hình của docker compose để thêm dịch vụ vào. Nội dung file `docker-compose.yml` (cần bổ sung):

```
services:
  # ... (các service cũ: wordpress, mysql, prometheus, grafana, loki, promtail...)

  alertmanager:
    image: prom/alertmanager:latest
    container_name: alertmanager
    restart: unless-stopped
    ports:
      - "9093:9093"
    volumes:
      - ./alertmanager.yml:/etc/alertmanager/config.yml
      - alertmanager_data:/alertmanager
    command:
      - '--config.file=/etc/alertmanager/config.yml'
      - '--storage.path=/alertmanager'
    networks:
      - monitoring
    depends_on:
      - prometheus

volumes:
  alertmanager_data:
    # ... các volume cũ
```

Để cấu hình dịch vụ Alertmanager, ta cần tạo file `alertmanager.yml` để lưu các thông tin thiết lập như sau:

Tạo file `alertmanager.yml`:

```
global:
  resolve_timeout: 5m

route:
  group_by: ['alertname']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  receiver: 'default' # Thay đổi theo kênh bạn muốn

receivers:
  - name: 'default'
    email_configs:
      - to: 'your-email@gmail.com'
        from: 'alert@yourdomain.com'
        smarthost: 'smtp.gmail.com:587'
        auth_username: 'your-email@gmail.com'
        auth_identity: 'your-email@gmail.com'
        auth_password: 'your-app-password' # Dùng App Password nếu là Gmail
    # Hoặc dùng Slack:
    # slack_configs:
    #   - api_url: 'https://hooks.slack.com/services/xxx/xxx/xxx'
    #     channel: '#alerts'
```

```
inhibit_rules:
  - source_match:
      severity: 'critical'
    target_match:
      severity: 'warning'
    equal: ['alertname']
```

Cần thay đổi các thông số tài khoản trong file cấu hình, hoặc nếu dùng các dịch vụ thông báo như Slack, thì cần tìm hiểu thêm về cách tạo webhook trên Slack để nhận thông báo.

Tiếp theo, ta cần cập nhật thông tin cấu hình cho Prometheus để có thể nhận thông tin và tích hợp với Alertmanager. File `prometheus.yml` được chỉnh sửa bổ sung:

```
global:
  scrape_interval: 15s

alerting:
  alertmanagers:
    - static_configs:
      - targets:
        - 'alertmanager:9093'

rule_files:
  - "/etc/prometheus/rules/*.yml" # Thư mục chứa alerting rules

scrape_configs:
  # ... (giữ nguyên các job cũ)
```

Trong phần cấu hình bổ sung có đề cập đến các tập tin rules của Alertmanager, ta cần tạo thư mục `./rules` và file ví dụ `rules/alerts.yml`:

```
groups:
- name: wordpress-mysql-alerts
  rules:
    # WordPress Alerts
    - alert: WordPressHighCPU
      expr: rate(container_cpu_usage_seconds_total{container="wordpress"}[5m]) * 100 > 80
      for: 2m
      labels:
        severity: warning
      annotations:
        summary: "WordPress CPU cao"
        description: "CPU usage của WordPress > 80% trong 2 phút"

    - alert: WordPressHighMemory
      expr: container_memory_usage_bytes{container="wordpress"} / container_spec_memory_limit_bytes{container="wordpress"} * 100 > 85
      for: 2m
      labels:
        severity: critical
      annotations:
        summary: "WordPress Memory cao"

    # MySQL Alerts
```

```
- alert: MySQLTooManyConnections
  expr: mysql_global_status_threads_connected > 80
  for: 1m
  labels:
    severity: warning
  annotations:
    summary: "MySQL có quá nhiều kết nối"

- alert: MySQLErrorRateHigh
  expr: rate(mysql_global_status_errors_total[5m]) > 5
  for: 2m
  labels:
    severity: critical
  annotations:
    summary: "MySQL có nhiều lỗi"

# cAdvisor / Host
- alert: HighDiskUsage
  expr: (node_filesystem_avail_bytes / node_filesystem_size_bytes) * 100 < 15
  for: 5m
  labels:
    severity: warning
```

Sau khi thiết lập xong các tập tin cấu hình cho dịch vụ, ta tiến hành khởi động lại docker compose như sau:

```
$docker compose down
$docker compose up -d
```

Kiểm tra các container có hoạt động đúng trên các cổng dịch vụ như đã được thiết lập không. Một số URL được sử dụng:

- Alertmanager UI: <http://localhost:9093>
- Trong Prometheus UI (<http://localhost:9090>): chọn Alerts để xem rule đang active.
- Trong Grafana: thêm Alerting rule (có thể tạo alert trực tiếp trong Grafana từ Prometheus datasource).

Câu hỏi thảo luận

Câu 1: Trong hệ thống vừa triển khai, Promtail thu thập log và đẩy về Loki, trong khi Prometheus thu thập metrics. Theo em, tại sao việc tách biệt Metrics (Prometheus) và Logs (Loki) lại mang lại lợi ích? Hãy phân tích ưu nhược điểm của cách tiếp cận này so với việc dùng một công cụ duy nhất quản lý cả metrics và logs.

Câu 2: Promtail đang thu thập log từ Docker containers thông qua Docker socket. Nếu hệ thống có hàng trăm container chạy trên nhiều host khác nhau, em sẽ thiết kế cách thu thập log như thế nào để đảm bảo hiệu suất và độ tin cậy? Promtail có những hạn chế gì và Grafana Alloy có thể giải quyết vấn đề đó ra sao?

Câu 3: Bạn đã thiết lập một số alert rule trong Prometheus và gửi thông báo qua Alertmanager. Theo bạn, nên thiết kế chiến lược alerting như thế nào để tránh tình trạng "alert fatigue" (quá nhiều cảnh báo gây mệt mỏi)? Đưa ra các best practices về grouping, routing, inhibition và silence mà bạn đã học hoặc tìm hiểu.

Câu 4: Sau khi hoàn thành bài thực hành với Prometheus + Grafana + Loki + Promtail + Alertmanager, em đánh giá stack này (thường gọi là Grafana Stack hoặc Loki + Prometheus + Grafana) có phù hợp để triển khai cho môi trường production không? Những thách thức lớn nhất khi scale hệ thống này lên nhiều server/node là gì và bạn sẽ giải quyết chúng như thế nào?